RICE UNIVERSITY

Collective Communication

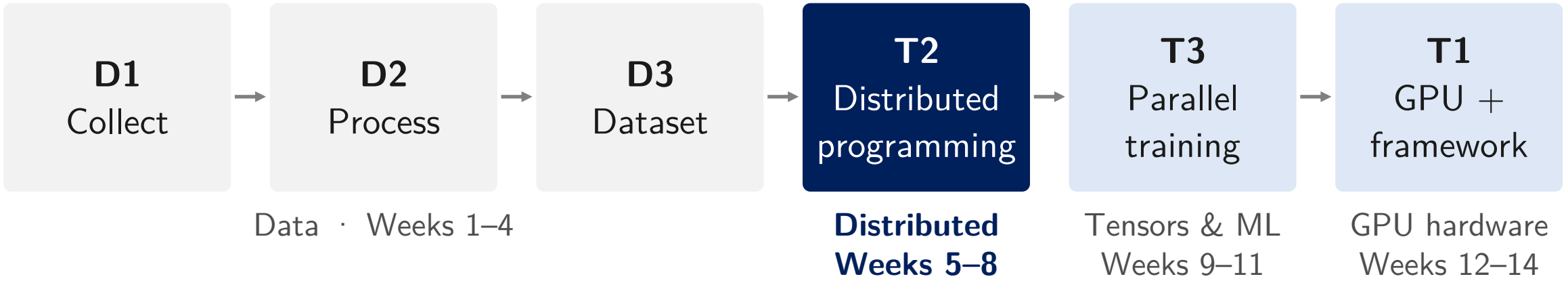
Share, collect, and sum: the standard collectives, all-reduce in two steps, and trees

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 7, Lecture 2

Where we are in the pipeline



In our pipeline

D3 due Friday · capstone spec out this week · **T2 and Lab 4 out after today** · midterm next week (Weeks 1–7).

Week 7 L1: Send, Recv → today: collectives → Week 8: rings, trees, and NCCL

Today's plan

1

The standard collectives

What does each one leave on every rank?

2

Building all-reduce

Why build it from reduce-scatter and all-gather?

3

Trees and their cost

How can p ranks finish in about $\log_2$ p steps?

Build

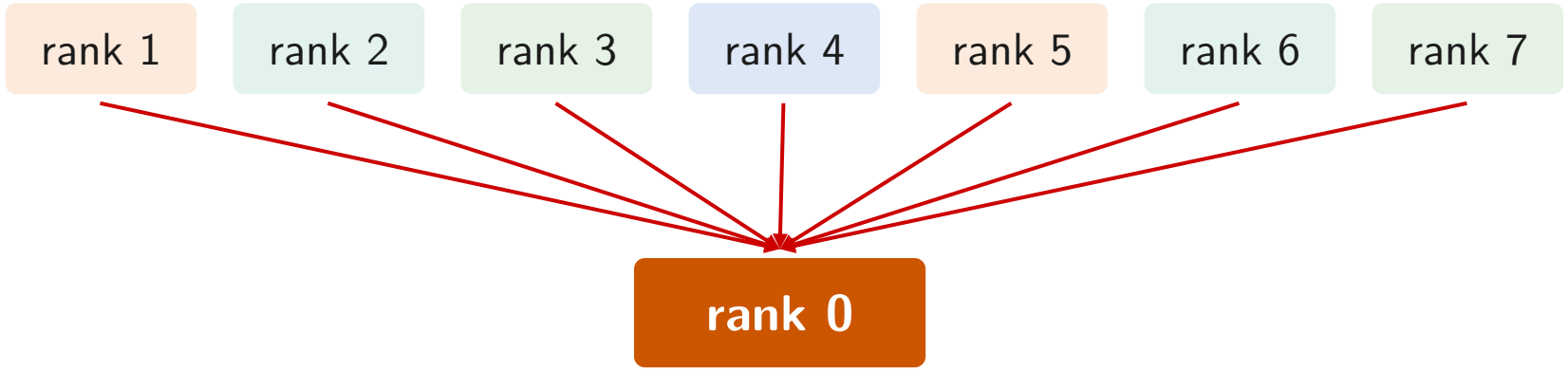
All-reduce in two calls; a tree's cost

Pairs: write it, check it against Allreduce, then count a tree by hand.

Why do we need collectives?

8 processes each hold 1 GB of gradients. All 8 need the **sum**. You have only Send and Recv.

How would you write it? Who does the most work?



The obvious way

14 messages, all through rank 0: it alone receives **7 GB**, one array after another.

Answer

A **collective** does it in **one call**, with an algorithm that keeps every link busy.

Source: Counts derived; 1 GB per rank and p = 8 are illustrative.

What a collective is, in plain words

collective: one operation that **every** process in a group calls together, to share, collect, or sum data.

your program	comm.Allreduce(g, total)	today
collective	many messages, arranged by the library	today
point-to-point	Send / Recv between two ranks	Week 7 L1
network	$\alpha + n/\beta$ per message	Week 6

Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023), Chapter 6; mpi4py documentation.

Where collectives show up

Collective	Used for	In this course
all-reduce	averaging gradients (DDP)	Week 9; T2, T3
all-gather, reduce-scatter	split training state (ZeRO, FSDP)	Week 9 L2
broadcast	same starting weights everywhere	Week 9; Lab 4

Why we need it

Training programs are written in a **few group operations**. A library does each in one call, faster and safer than hand-written messages.

Source: MPI 4.1 standard; NVIDIA NCCL User Guide; PyTorch Distributed Overview; ZeRO, SC 2020.

Learning goals for today

- **Draw** what each standard collective leaves on every rank.
- **Build** all-reduce from reduce-scatter and all-gather, and say why splitting helps.
- **Count** the cost of a tree broadcast, reduce, and all-reduce with α and β.
- **Call** collectives from mpi4py and **check** a result against a reference.

A puzzle to start

Rank 0 has 100 MB of model weights.
63 other ranks need a copy.
One copy over one link takes **2 ms**.

Sending them one by one takes 63 × 2 ms ≈ 126 ms. Can you do much better?

Answer

Yes: let every rank that has a copy **forward it**. 2, 4, 8, ... 64 ranks: **6 steps**, ≈ **12 ms**.

Source: 400 Gb/s NDR InfiniBand NIC = 50 GB/s per direction (typical in H100 servers); $\alpha = 5$ µs assumed.

Part 1

The standard collectives

What does each one leave on every rank?

1 · The standard collectives

2 · Building all-reduce

3 · Trees and their cost

Build · All-reduce in two calls; a tree's cost

What is a collective?

- A **collective** is called by **every** rank of the group, in the **same order**.
- Some have a **root**: the one rank that provides the data or receives the result.
- A **reduction** combines the ranks' arrays elementwise, e.g. by **sum**.

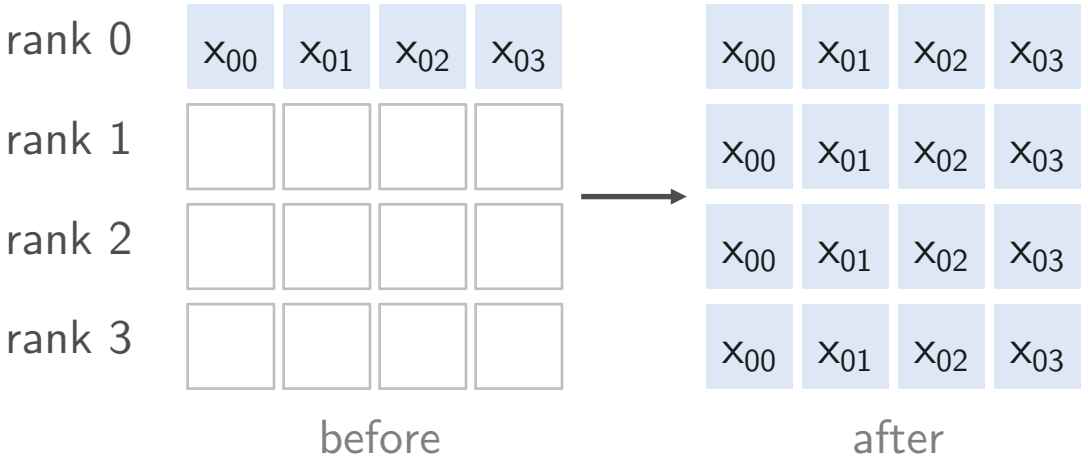
Watch out

If one rank skips or reorders a collective, the others wait **forever**.

Source: MPI standard 4.1, Chapter 6.

Broadcast

Broadcast (root: rank 0)



the root's array is copied to everyone

Use

Start every copy of a model from the **same weights** (Week 9).

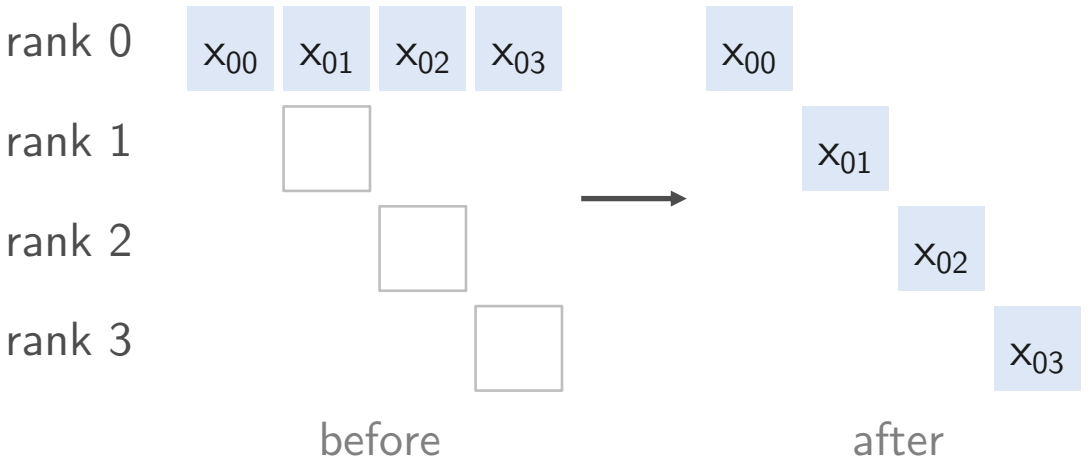
Key idea

Broadcast: every rank ends with **all** of the root's data.

Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023), Chapter 6; mpi4py documentation.

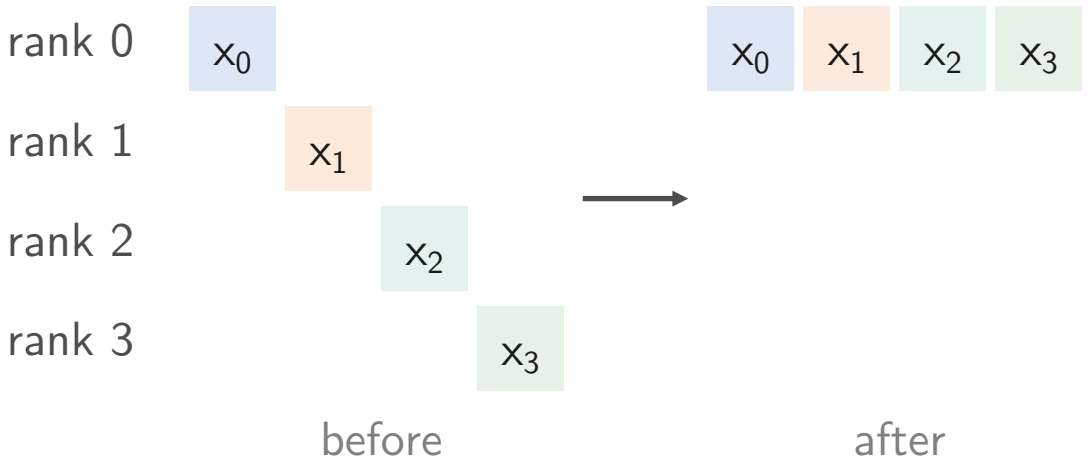
Scatter and gather

Scatter (root: rank 0)



rank i gets chunk i of the root's array

Gather (root: rank 0)



the root gets every rank's chunk

Key idea

Scatter **splits** the root's array across ranks; gather **joins** the pieces on the root.

Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023), Chapter 6; mpi4py documentation.

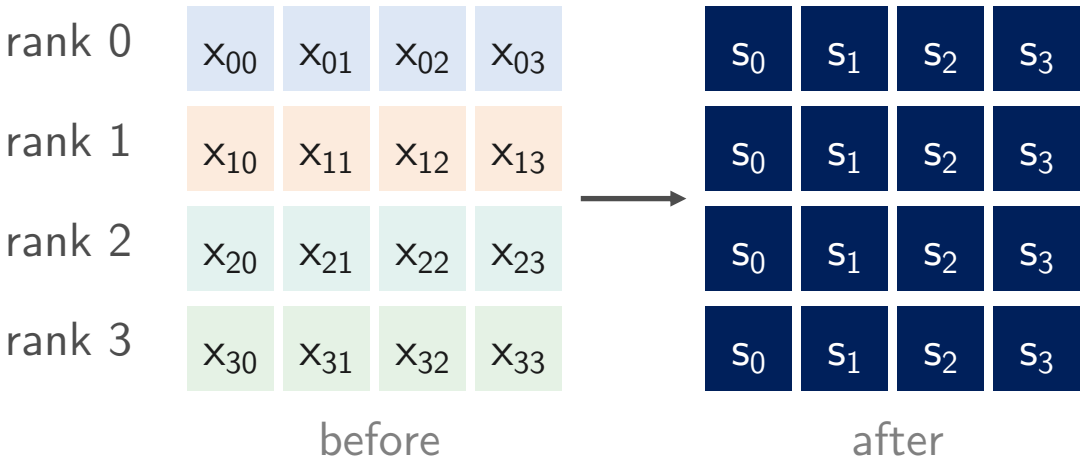
Reduce and all-reduce

Reduce, sum (root: rank 0)



the sum lands on the root only

All-reduce, sum



the sum lands on every rank

Key idea

$s_0 = x_{00} + x_{10} + x_{20} + x_{30}$, elementwise. All-reduce: the sum on **every** rank.

Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023), Chapter 6; mpi4py documentation.

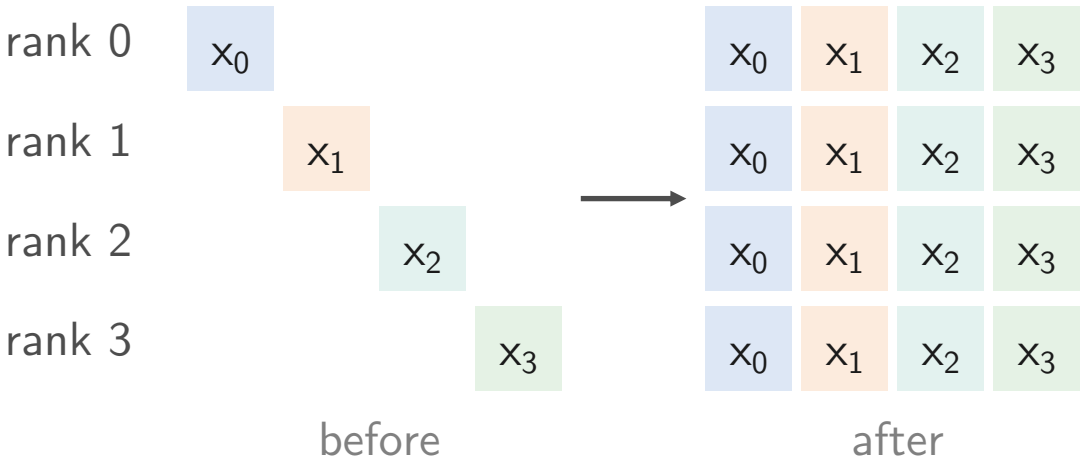
Reduce-scatter and all-gather

Reduce-scatter, sum



rank i keeps chunk i of the sum

All-gather



every rank gets every chunk

Key idea

Reduce-scatter: each rank ends with **one chunk** of the sum. All-gather: each rank ends with **all** chunks.

Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023), Chapter 6; mpi4py documentation.

Three shapes of collective

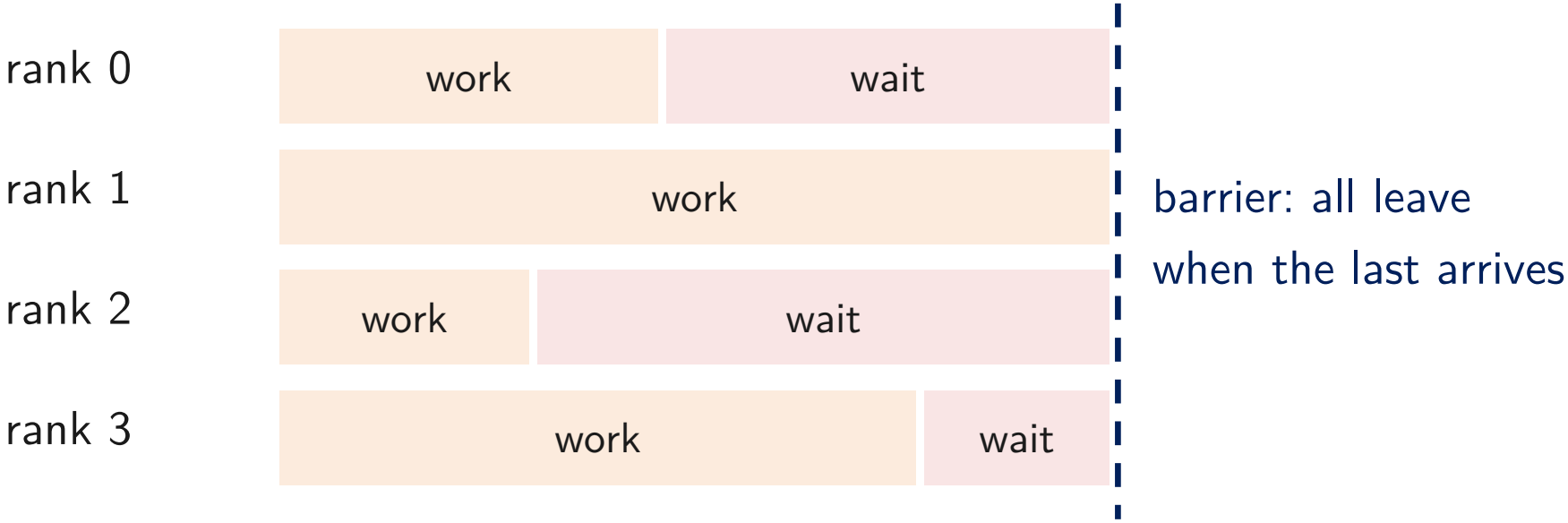
Shape	Move data only	Also sum
one $\rightarrow$ all	broadcast, scatter	—
all $\rightarrow$ one	gather	reduce
all $\rightarrow$ all	all-gather	all-reduce, reduce-scatter

Key idea

An **all-** collective is its rooted version with the result on **every** rank.

Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023), Chapter 6; mpi4py documentation.

Synchronization: the barrier



Key idea

A **barrier** lines everyone up — use it before starting a timer. Other collectives may not wait for everyone.

Source: MPI Forum, MPI: A Message-Passing Interface Standard, Version 4.1 (2023), Chapter 6; mpi4py documentation.

Check yourself

Two minutes with a neighbour.

- Rank 0 read a config file that every rank needs. Which collective?
- Four ranks, each with a 1,000-element array, call reduce-scatter. What does each get?
- Ranks 0, 1, 3 call all-reduce; rank 2 calls broadcast first. What happens?

Part 2

Building all-reduce

Why build it from reduce-scatter and all-gather?

1 · The standard collectives

2 · Building all-reduce

3 · Trees and their cost

Build · All-reduce in two calls; a tree's cost

A first use: averaging gradients

```
g = local_gradient()        # my share of the batch
total = np.empty_like(g)
comm.Allreduce(g, total, op=MPI.SUM)
g_avg = total / p           # same on every rank
```

average of the ranks' gradients

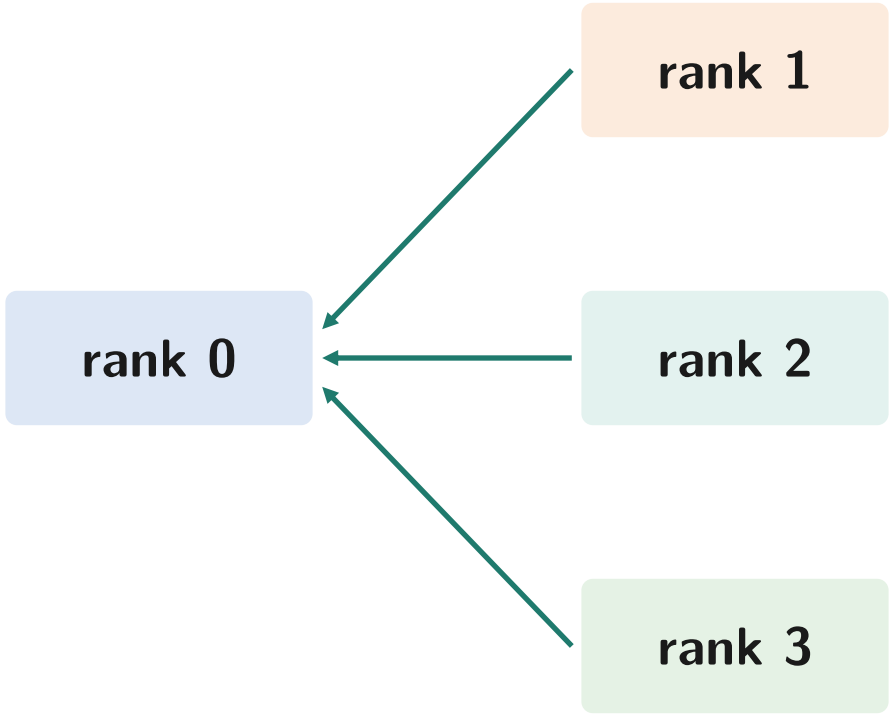
$=$ gradient of the average loss

In our pipeline

T2 builds this all-reduce from **Send** and **Recv**; T3 trains a model with it.

Source: MPI standard 4.1, Section 6.9; Li et al., "PyTorch Distributed," VLDB 2020.

A naive all-reduce



$$T = 2\,(p-1)(\alpha + n\,/\,\beta)$$

Watch out

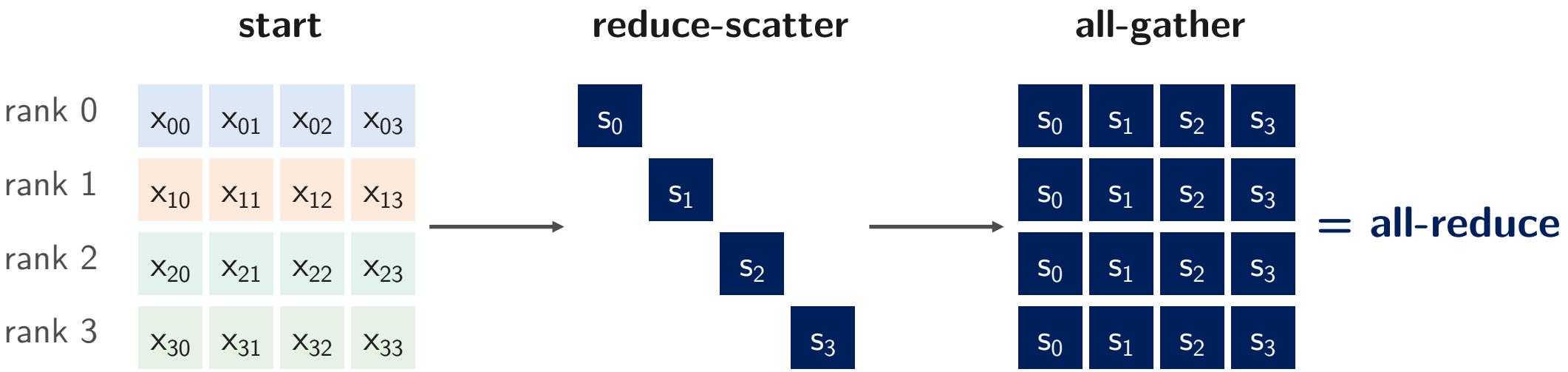
p = 8, 1 GB on NVLink: $\approx$ **31 ms**. Rank 0's link carries **everything**: time grows **with p**.

p − 1 arrays in, one after another;

then p − 1 sums back out

Source: H100 datasheet: NVLink 4, 900 GB/s bidirectional per GPU = 450 GB/s per direction; idealized.

All-reduce = reduce-scatter + all-gather



Key idea

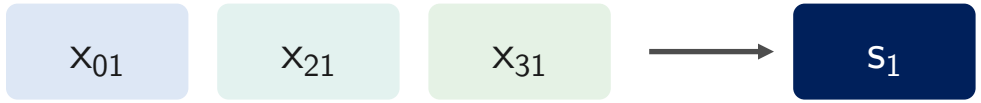
Step 1 leaves **one chunk of the sum** on each rank; step 2 hands every chunk to everyone.

Source: Thakur, Rabenseifner & Gropp, "Optimization of Collective Communication Operations in MPICH," IJHPCA 19(1), 2005.

Why two steps? Every rank shares the work

rank 0 sums chunk 0 | **rank 1** sums chunk 1 | **rank 2** sums chunk 2 | **rank 3** sums chunk 3

rank 1 receives chunk 1 from ranks 0, 2, 3:



Into the busiest rank ($n = 1$ GB, $p = 8$)	**Bytes**
naive: rank 0 receives $(p-1) \cdot n$	7 GB
reduce-scatter: each rank receives $(p-1) \cdot n/p$	0.875 GB

Key idea

No rank is a bottleneck. **How** to schedule the pieces fast is Week 8 (the ring).

Source: Counts derived; Thakur, Rabenseifner & Gropp, "Optimization of Collective Communication Operations in MPICH," IJHPCA 19(1), 2005.

Reduce-scatter and all-gather on their own

Collective	When you need it	Where
reduce-scatter	each rank needs only its slice of a sum	Week 9 L2
all-gather	rebuild an array split across ranks	Weeks 9–10
both, in a row	everyone needs the whole sum	all-reduce

In our pipeline

Learn these two well: split-state training (ZeRO, FSDP) uses them **instead of** all-reduce.

Source: Rajbhandari et al., ZeRO, SC 2020; PyTorch FSDP documentation.

Check yourself

Two minutes with a neighbour.

- All-reduce $=$? $+$?. After the first step, what does rank 2 hold?
- Naive all-reduce, p $= 16$, n $= 1$ GB: how many message times? Bytes into rank 0?
- p $= 8$, n $= 1$ GB: after a reduce-scatter, how much of the sum does each rank hold?

Part 3

Trees and their cost

How can p ranks finish in about $\log_2$ p steps?

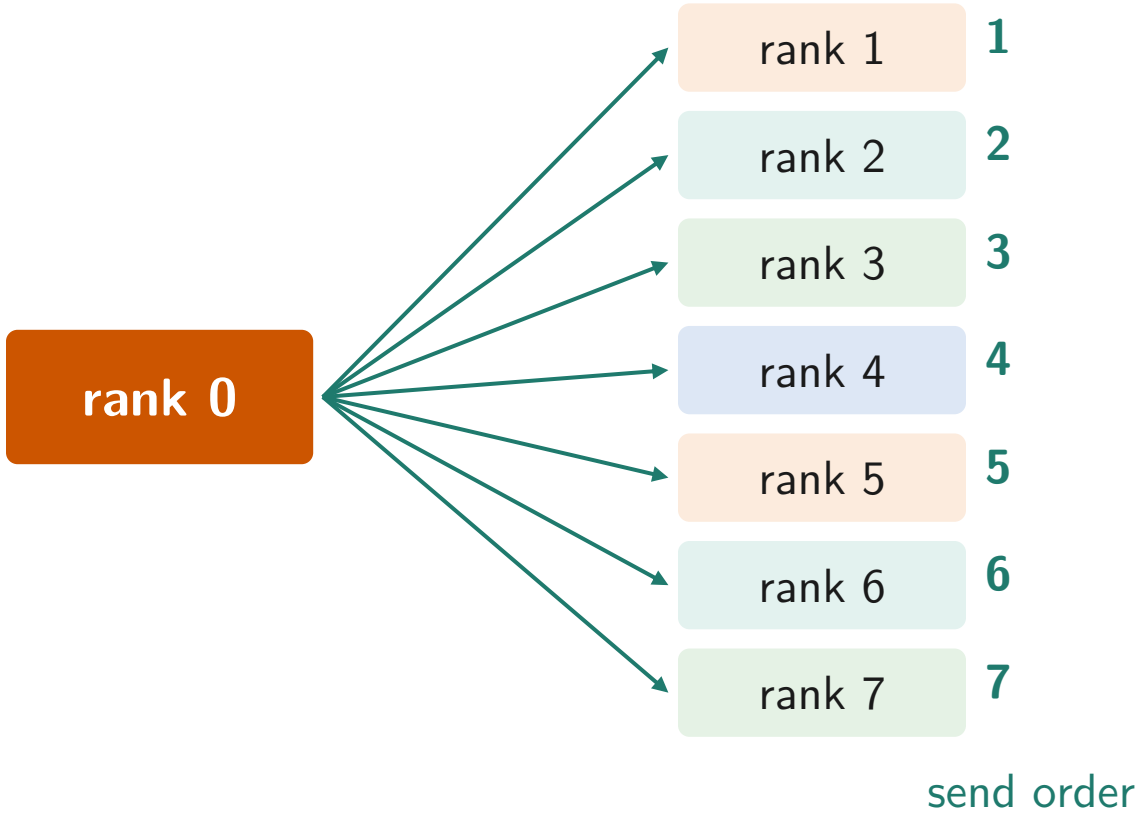
1 · The standard collectives

2 · Building all-reduce

3 · Trees and their cost

Build · All-reduce in two calls; a tree's cost

Broadcast by a loop



$$T = (p - 1)(\alpha + n / \beta)$$

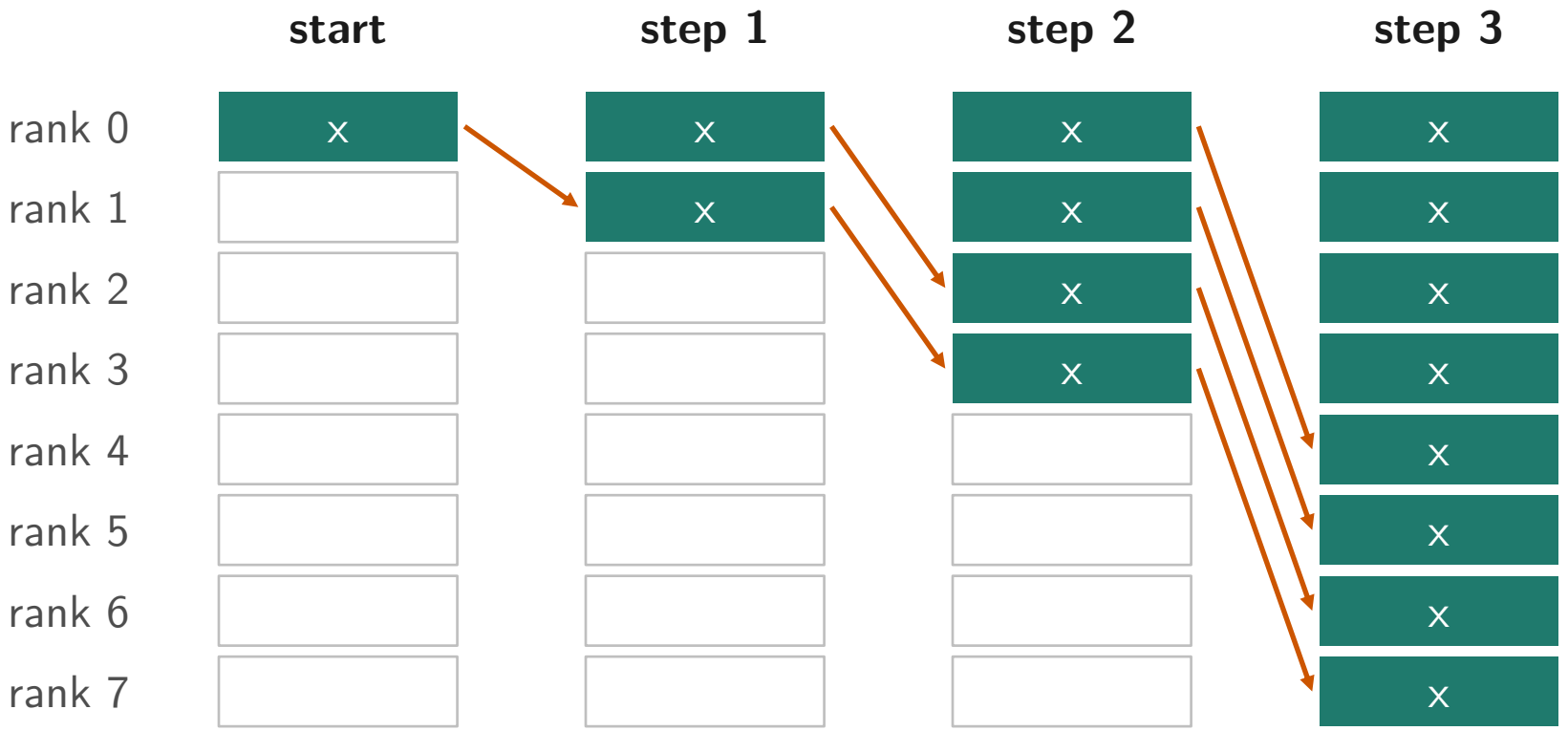
p = 8: **7** message times, one after another

Watch out

One link does all the sending; the others wait. Time grows **linearly** in p.

Source: Thakur, Rabenseifner & Gropp, "Optimization of Collective Communication Operations in MPICH," IJHPCA 19(1), 2005.

A tree: everyone who has it, forwards it



x = has the data

ranks with the data:

$1 \to 2 \to 4 \to 8$

Step cost

Each step: **one** message time; the sends use **different** links.

Source: Thakur, Rabenseifner & Gropp, "Optimization of Collective Communication Operations in MPICH," IJHPCA 19(1), 2005.

Counting the tree

ranks with the data double each step: 1, 2, 4, 8, ...

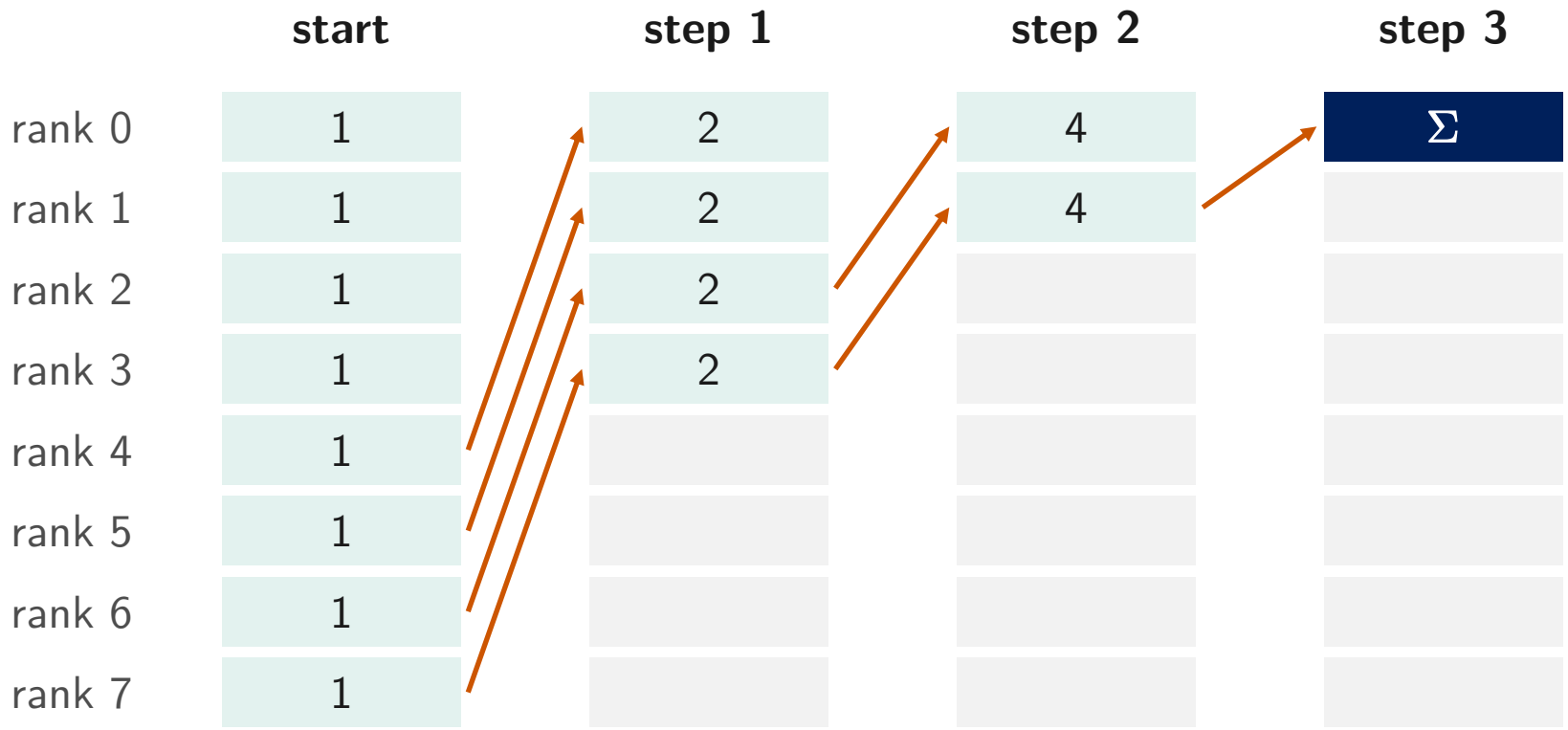
k doublings reach p ranks when $k = \lceil \log_2 p \rceil$

$$T(\text{broadcast}) = \lceil \log_2 p \rceil \cdot (\alpha + n / \beta)$$

Ranks p	Loop: p − 1	Tree: $\lceil \log_2 p \rceil$
8	7	3
64	63	6
1,024	1,023	10

Source: Thakur, Rabenseifner & Gropp, "Optimization of Collective Communication Operations in MPICH," IJHPCA 19(1), 2005.

Reduce: the same tree, upside down



number = how many ranks' arrays are summed in

$$T(\text{reduce}) = \lceil \log_2 p \rceil \, (\alpha + n/\beta)$$

Source: Thakur, Rabenseifner & Gropp, "Optimization of Collective Communication Operations in MPICH," IJHPCA 19(1), 2005.

All-reduce on a tree

$$\text{all-reduce} = \text{reduce} + \text{broadcast}$$

$$T(\text{tree all-reduce}) = 2\,\lceil \log_2 p \rceil \cdot (\alpha + n\,/\,\beta)$$

$$\text{naive:}\quad 2(p-1) \cdot (\alpha + n\,/\,\beta)$$

p = 64: **12** message times instead of **126**

Watch out

Each tree step still moves the **whole** array; when that hurts is **Week 8** (the ring).

Source: Thakur, Rabenseifner & Gropp, "Optimization of Collective Communication Operations in MPICH," IJHPCA 19(1), 2005.

Back to the puzzle

$$\text{loop:}\quad 63 \times 2\ \text{ms} \approx 126\ \text{ms}$$

$$\text{tree:}\quad \lceil \log_2 64 \rceil = 6\ \text{steps},\quad 6 \times 2\ \text{ms} \approx 12\ \text{ms}$$

Answer

Same bytes in total, **10× less time**: the tree uses many links **at once**.

Source: 400 Gb/s NDR InfiniBand NIC = 50 GB/s per direction (typical in H100 servers); $\alpha = 5$ µs assumed.

Check yourself

Two minutes with a neighbour.

- How many steps does a tree broadcast to $p = 1{,}000$ ranks take?
- Loop or tree broadcast: which sends more bytes in total? Then why is the tree faster?
- Tree reduce on 8 ranks: how many additions does rank 0 do?

IN-CLASS BUILD

All-reduce in two calls; a tree's cost

Pairs: write it, check it against Allreduce, then count a tree by hand.

1 · The standard collectives

2 · Building all-reduce

3 · Trees and their cost

Build · All-reduce in two calls; a tree's cost

Exercise 1: all-reduce in two calls

```
n = 1000 * p                        # divisible by p
rng = np.random.default_rng(rank)
g = rng.standard_normal(n)
part = np.empty(n // p)
# 1. reduce-scatter g into part
# 2. all-gather part into total
total = np.empty(n)
```

DO

1. Fill in the two calls.
2. Compare with `Allreduce`.
3. Run on 4 processes.

ASK

Compare with `np.allclose`, not ==. Why?

Source: mpi4py documentation; MPI standard 4.1, Chapter 6.

Exercise 2: count a tree

$$\mathbf{p = 16}\ \text{ranks},\quad \mathbf{n = 4\ MB}$$
$$\alpha = 5\ \mu\text{s},\quad \beta = 50\ \text{GB/s}$$

- Time of **one** message, $\alpha + n/\beta$?
- Broadcast: loop versus tree. Steps and time?
- Tree all-reduce: steps and time?
- With $p = 12$ instead: how many tree steps?

STRETCH

Draw the broadcast tree for $p = 6$. Which ranks send in the last step?

Source: 400 Gb/s NDR InfiniBand NIC = 50 GB/s per direction (typical in H100 servers); $\alpha = 5$ µs assumed.

Solutions 1: the code

```
comm.Reduce_scatter_block(g, part, op=MPI.SUM)
comm.Allgather(part, total)
ref = np.empty(n)
comm.Allreduce(g, ref, op=MPI.SUM)
assert np.allclose(total, ref)
```

Why allclose

Rounding: the sums are added in a different order, so the last bits may differ. On one laptop run: `allclose` true, == false.

Source: mpi4py documentation; laptop test run, 4 processes (MPICH 5.0.2, mpi4py 4.1.2).

Solutions 2: the counts

one message: $5\ \mu s + 80\ \mu s = \mathbf{85\ \mu s}$

p = 16	**Steps**	**Time**
broadcast, loop	15	1,275 µs
broadcast, tree	4	340 µs
all-reduce, tree	8	680 µs

Check

p = 12 also takes **4** steps: $\lceil \log_2 12 \rceil = 4$. Stretch: in step 3 only ranks 0 and 1 send.

Source: 400 Gb/s NDR InfiniBand NIC = 50 GB/s per direction (typical in H100 servers); $\alpha = 5$ µs assumed.

How to test a collective

- Compare with a **reference**: `Allreduce`, or a sum on one process.
- Give each rank **different** data (seed = rank).
- Run with p = 1, 2, 3, 4, 6: odd p and non-powers of two.
- Try a length p does **not** divide.

In our pipeline

T2 grades **correctness** first: a fast all-reduce with a wrong chunk is worth nothing.

Source: mpi4py documentation; MPI standard 4.1, Chapter 6.

Key takeaways

- A **collective** is called by every rank, in the same order; skip one and everyone hangs.
- Seven shapes: broadcast, scatter, gather, reduce, all-gather, all-reduce, reduce-scatter.
- **All-reduce = reduce-scatter + all-gather**: every rank shares the work.
- Tree broadcast or reduce: $\lceil \mathbf{\log_2 p} \rceil\ (\alpha + \mathbf{n}/\beta)$; a tree all-reduce costs twice that.
- Test against a **reference** with `allclose`, on several p.

Midterm review: Weeks 5–7

Week	Be able to
5	processes vs threads · Amdahl's law · races, locks, deadlock
6	$\alpha + \beta n$ and its crossover · TCP vs UDP · RPC · RDMA
7	blocking vs non-blocking · the collectives · tree cost

In our pipeline

Midterm next week covers **Weeks 1–7**. Redo every "Check yourself" — the questions are in that style.

Next time, and readings

When	What
Today	T2 and Lab 4 released
Friday (this week)	D3 due; capstone spec out this week
Week 8, Lecture 1	All-reduce: rings and trees
Week 8	Midterm (Weeks 1–7)

READINGS

Thakur, Rabenseifner & Gropp, "Optimization of Collective Communication Operations in MPICH," IJHPCA 2005 (Sections 1–4).

MPI standard 4.1, Chapter 6, or the mpi4py tutorial.